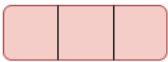
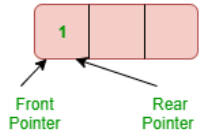


DS	Session	Queue	Question Information	Level 1	Challenge 58
Question description Selvan is very interested in surfing the contents from google. He searches for various coding test on Google. One day he searched about online coding competitions, in the retrieval links, he received many links for coding competition. he chooses first link from the goole suggestion list. first question for the coding competition is LRU cache implementation using queue concepts. Function Description 1. Queue which is implemented using a doubly linked list. The maximum size of the queue will be equal to the total number of frames available (cache size). The most recently used pages will be near front end and least recently pages will be near the rear end. 2. A Hash with page number as key and address of the corresponding queue node as value.					
CH.SC.U4CSE25225 - M ASHWIN Department of Computer Science & Engineering (CSE) 2026					

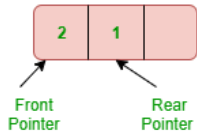
Given 3 page frames, so we take size of Queue is 3. Initially, Queue is empty.



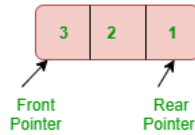
Input : 1



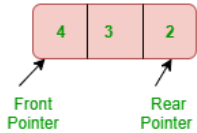
Input : 2 (every new input will be front as defined LRU)



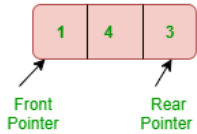
Input : 3



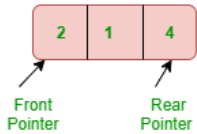
Input : 4 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



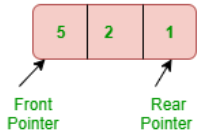
Input : 1 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 2 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Input : 5 (since all the frames are full, we remove a node from the rear of queue, and add the new node to the front of queue.)



Constraints

For this experiment, cache can hold 4 pages.

Let 10 different pages can be requested (pages to be referenced are numbered from 0 to 9).

Input Format:

First line represents n and m, where n is the page number with in the range (0-9) & m is cache size (must be 4 for this problem).

Next line represents the reference pages.

Output Format:

Single line represents the cache frames after the above referenced pages.

Test Case 1

INPUT (STDIN)

```
6 4
1 2 3 1 4 5
```

EXPECTED OUTPUT

```
5 4 1 3
```

Test Case 2

INPUT (STDIN)

```
10 4
3 1 4 5 9 8 7 6 1 2
```

EXPECTED OUTPUT

```
2 1 6 7
```

▼ Mandatory Test Cases

Test Case 1

KEYWORD

```
QNode* newNode(unsigned pageNumber)
```

Test Case 2

KEYWORD

```
Queue* createQueue(int numberOfFrames)
```

Test Case 3

KEYWORD

```
typedef struct Queue
```

Test Case 4

KEYWORD

```
typedef struct QNode
```

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

```
1
```

Test Case 2

TOKEN COUNT

```
750
```

Test Case 3

NLOC

```
120
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct QNode {
5     unsigned pageNumber;
6     struct QNode *prev, *next;
7 } QNode;
8
9 typedef struct Queue {
10     unsigned count;
11     unsigned numberOfFrames;
12     QNode *front, *rear;
13 } Queue;
14
15 QNode* newQNode(unsigned pageNumber) {
16     QNode* temp = (QNode*)malloc(sizeof(QNode));
17     temp->pageNumber = pageNumber;
18     temp->prev = temp->next = NULL;
19     return temp;
20 }
21
22 Queue* createQueue(int numberOfFrames) {
23     Queue* queue = (Queue*)malloc(sizeof(Queue));
24     queue->count = 0;
25     queue->front = queue->rear = NULL;
```

```

26 queue->numberOfFrames = numberOfFrames;
27 return queue;
28}
29
30void removeNode(Queue* queue, QNode* node) {
31     if (node->prev != NULL) {
32         node->prev->next = node->next;
33     } else {
34         queue->front = node->next;
35     }
36     if (node->next != NULL) {
37         node->next->prev = node->prev;
38     } else {
39         queue->rear = node->prev;
40     }
41     queue->count--;
42}
43
44void addToFront(Queue* queue, QNode* node) {

```

```

45     node->next = queue->front;
46     node->prev = NULL;
47     if (queue->front != NULL) {
48         queue->front->prev = node;
49     }
50     queue->front = node;
51     if (queue->rear == NULL) {
52         queue->rear = node;
53     }
54     queue->count++;
55 }
56
57 void referencePage(Queue* queue, unsigned pageNumber) {
58     QNode* curr = queue->front;
59     QNode* found = NULL;
60     while (curr != NULL) {
61         if (curr->pageNumber == pageNumber) {
62             found = curr;
63             break;

```

```
64     }
65     curr = curr->next;
66 }
67
68 if (found != NULL) {
69     removeNode(queue, found);
70     addToFront(queue, found);
71 } else {
72     if (queue->count == queue->numberOfFrames) {
73         QNode* lru = queue->rear;
74         removeNode(queue, lru);
75         free(lru);
76     }
77     QNode* newNode = newQNode(pageNumber);
78     addToFront(queue, newNode);
79 }
80}
81
82int main() {
```

```

83  int n, k;
84  if (scanf("%d %d", &n, &k) != 2) return 0;
85
86  Queue* queue = createQueue(k);
87
88  for (int i = 0; i < n; i++) {
89      unsigned page;
90      scanf("%u", &page);
91      referencePage(queue, page);
92  }
93
94  QNode* curr = queue->front;
95  while (curr != NULL) {
96      printf("%u%c", curr->pageNumber, (curr->next == NULL) ? '\n' : ' ');
97      curr = curr->next;
98  }
99  return 0;
100
101

```

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t1.txt

6 4

1 2 3 1 4 5

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue8.exe

5 4 1 3

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>type t2.txt

10 4

3 1 4 5 9 8 7 6 1 2

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>Queue8.exe

2 1 6 7

C:\Users\CH.SC.U4CSE25225\Desktop\DSA Lab>